

Project 1

N-Body Problem

September 14, 2026

PROJECT ASSIGNMENT: GRAVITATIONAL N-BODY PROBLEM AND ORBITAL STABILITY

PROJECT OVERVIEW AND LEARNING OBJECTIVES

The gravitational N-body problem represents a cornerstone of computational astrophysics, challenging our ability to model the evolution of dynamical systems where analytical solutions are non-existent for $N > 2$. This project is designed to bridge the gap between the elegant theoretical frameworks of analytic celestial mechanics and the rigorous requirements of numerical simulation. Students will move from the mathematical foundations of finite difference approximations to the deployment of high-fidelity integrators capable of propagating complex orbital architectures.

The core objective is the development of a proprietary simulation engine to explore the stability and precession of multi-body systems. The project is structured into four distinct phases:

1. Mathematical Formalism and Numerical Differentiation: Establishing the derivative-based foundations of motion.
2. Algorithmic Implementation and ODE Solvers: Architecting the propagation engine and event-detection logic.
3. Multi-Stage Simulation Tasks: Benchmarking against analytic baselines and expanding to 3D N-body systems.
4. Physical Validation and Stability Analysis: Quantifying energy conservation and the relationship between step size and global truncation error.

PHASE I: MATHEMATICAL FORMALISM AND NUMERICAL DIFFERENTIATION

To propagate an N-body system, we must express the equations of motion in a 3D Cartesian coordinate system, where the acceleration of any body i is the vector sum of the gravitational gradients produced by all other bodies j .

DERIVATION TASK: LAGRANGE INTERPOLATION BASIS

Your first task is to justify the general formula for numerical differentiation using Lagrange's interpolation. Given a function $f(x)$ approximated by:

$$f(x) = \sum_{j=0}^n l_j(x) f_j \quad (0.1)$$

where

$$l_j(x) = \frac{\pi(x)}{(x - x_j)\pi'(x_j)}, \quad (0.2)$$

you must derive the General Formula for Numerical Differentiation (n points):

$$f'(x) = \frac{1}{h} \left[\frac{(-1)^n}{n! a_{0n}} \left(\sum_{k=1}^n k a_{0k} s^{k-1} \right) f_0 + \sum_{j=1}^n \left\{ \frac{(-1)^{n-j}}{j!(n-j)! a_{jn}} \left(\sum_{k=1}^n k a_{jk} s^{k-1} \right) \right\} f_j \right] \quad (0.3)$$

NUMERICAL DIFFERENTIATION REQUIREMENTS

During the validation phase of your simulation (Phase IV), you will use finite difference approximations to verify state vectors. You are required to implement the following formulas:

- General Three-Point First Derivative Formula (n=2):

$$f'(x) = \frac{(-3 + 2s)f_0 + 4(1 - s)f_1 + (-1 + 2s)f_2}{2h} \quad (0.4)$$

Note: Students must demonstrate that the standard central difference formula is a special case of Eq 0.4 where the parameter $s=1$.

- Second Derivative Difference Formula (n=2):

$$f''(x) = \frac{f_0 - 2f_1 + f_2}{h^2} \quad (0.5)$$

PHASE II: ALGORITHMIC IMPLEMENTATION AND ODE SOLVERS

You are commanded to develop a modular simulation environment from the ground up. The use of external ODE libraries is not allowed.

MANDATORY SOLVERS

The core integration loop must utilize one of the following:

- 4th-Order Runge-Kutta (RK4): Balancing computational overhead with high-order local accuracy.
- Symplectic Integrator: Preferred for long-duration stability and Hamiltonian preservation.

EVENT DETECTION AND DATA REFINEMENT

- Bisection Method: Implement a bisection routine to identify orbital crossing points or apsides. For verification of your root-finding logic, ensure your solver can identify the specific crossing constant of 2.667 in a unit-normalized test case.
- Lagrange Interpolation: Use the methodology from Section 2 of Phase I to interpolate state data between discrete timesteps. This is critical when the integration step h is too coarse to capture close-encounter dynamics.

CODE REQUIREMENTS CHECKLIST

- Modularity: Encapsulate gravitational potential logic separately from the integrator.
- Vectorization: Use 3D vector operations for all position and velocity updates.
- Timestamped Logging: Record simulation time, state vectors, and total system energy at every step.

PHASE III: MULTI-STAGE SIMULATION TASKS

TASK 1: THE ANALYTIC 2-BODY BENCHMARK

Model the Earth and a specific satellite (e.g., the ISS or a GPS Block III) using real-world initial conditions.

- Requirement: Pull the semi-major axis, eccentricity, and inclination from recent TLE (Two-Line Element) data.
- Analysis: Compare your numerical output against the Keplerian analytic solution. Measure the “solver drift”—the cumulative deviation in position—over 100 orbital periods.

TASK 2: THE RESTRICTED 3-BODY SYSTEM

Model a system containing the Earth, the Moon, and a low-mass Satellite.

- Analysis: Map the trajectory of the satellite and numerically identify the L_4 or L_5 Lagrangian points or regions of orbital stability within the Earth-Moon gravitational well.

TASK 3: N-BODY EXPANSION (THE INNER SOLAR SYSTEM)

Include Mercury in a full 3D environment.

- Inclination Requirement: You must explicitly account for Mercury’s 7-degree orbital inclination relative to the ecliptic.
- Analysis: Analyse the resulting 3D orbital precession. Discuss the qualitative differences in long-term stability when comparing this 3D model to a simplified 2D planar approximation.

PHASE IV: PHYSICAL VALIDATION AND STABILITY ANALYSIS

Validation is the final arbiter of simulation quality. You must demonstrate that your numerical propagation respects the fundamental symmetries of the physical system.

CONSERVATION LAWS

Generate multi-panel Validation Graphs showing the evolution of:

1. Total Energy (E): Sum of kinetic and gravitational potential energy.
2. Total Angular Momentum (L): The system’s vector sum of angular momentum.

ERROR ANALYSIS AND STEP SIZE SELECTION

Conduct a formal analysis of how your chosen step size (h) influences global error. You must calculate the actual drift measured in Task 1 and compare it to the theoretical truncation error E'_n predicted by the following: General First Derivative Error (E'_n):

$$E'_n = \frac{h^n}{(n+1)!} \frac{f^{n+1}(\xi)}{a_{0n}} \left\{ \sum_{k=0}^n (k+1) a_{0k} s^k \right\} \quad (0.6)$$

Specific Error for $n=3$ (E'_3):

$$E'_3 = -\frac{h^3}{12} (3 - 11s + 9s^2 - 2s^3) f^{iv}(\xi) \quad (0.7)$$

Use these expressions to mathematically justify your selection of h for the N -body expansion.

SUBMISSION DELIVERABLES AND FORMATTING STANDARDS

Final submissions must be professional, technical, and complete. Required components include:

1. Documented Source Code: Original implementation of the RK4 or Symplectic solver.
2. Orbital Plots: 3D trajectory visualizations of the Sun-Mercury system, highlighting the 7-degree inclination.
3. Validation Suite: A figure layout containing Energy and Angular Momentum stability graphs.
4. Technical Report: A comprehensive synthesis comparing analytic and numerical results, including the formal comparison between measured solver drift and theoretical truncation error.

Final Constraint: All documentation must adhere to professional academic standards. Maintain technical rigour throughout and ensure all derivations are logically sequenced.